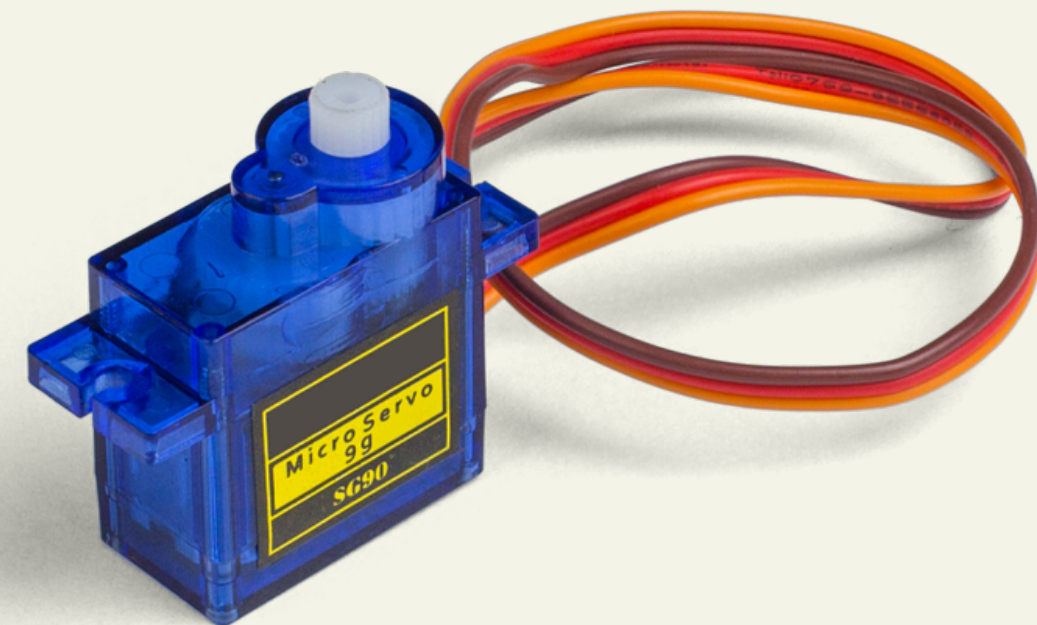


ARDUINO: CONCEITOS, PROGRAMAÇÃO E APLICAÇÃO PRÁTICA



AULA 03

COMUNICAÇÃO SERIAL, SENSORES E SERVO MOTOR



COMUNICAÇÃO SERIAL

- **Definição:**
 - Troca sequencial de dados entre dispositivos.
 - É como o Arduino "conversa" com outros dispositivos (computador, sensores, outros Arduinos).
- **Aplicações no Arduino:**
 - Ver valores de entradas e saídas por meio do monitor serial.
 - Enviar comandos do computador para o Arduino.
 - Conectar sensores (muitos sensores usam comunicação serial).

CONFIGURAÇÃO E BAUD RATE

- **Baud Rate (Taxa de Transmissão):**
 - **Número de bits por segundo (velocidade da comunicação).**
 - **Padrão para Arduino: 9600 bauds.**
- **Deve ser igual no Arduino e no Monitor Serial.**
- **Como iniciar a comunicação:**

```
void setup() {  
  Serial.begin(9600); // Taxa de 9600 bits por segundo  
}
```

FUNÇÕES BÁSICAS

- Para realizar os programas usando a comunicação serial, utilizam-se os comandos **print** e **read**:
 - **Função Serial.print:** Usada para envio de informações do Arduino para o PC.

```
Serial.print(Argumento); // Argumento pode ser texto (entre aspas) ou variável
```

- **Função Serial.println:** tem a mesma proposta da anterior. O seu diferencial é que após o argumento será acrescentada uma quebra de linha.
- **Função Serial.read:** É utilizada para o envio de informações do PC para o Arduino.

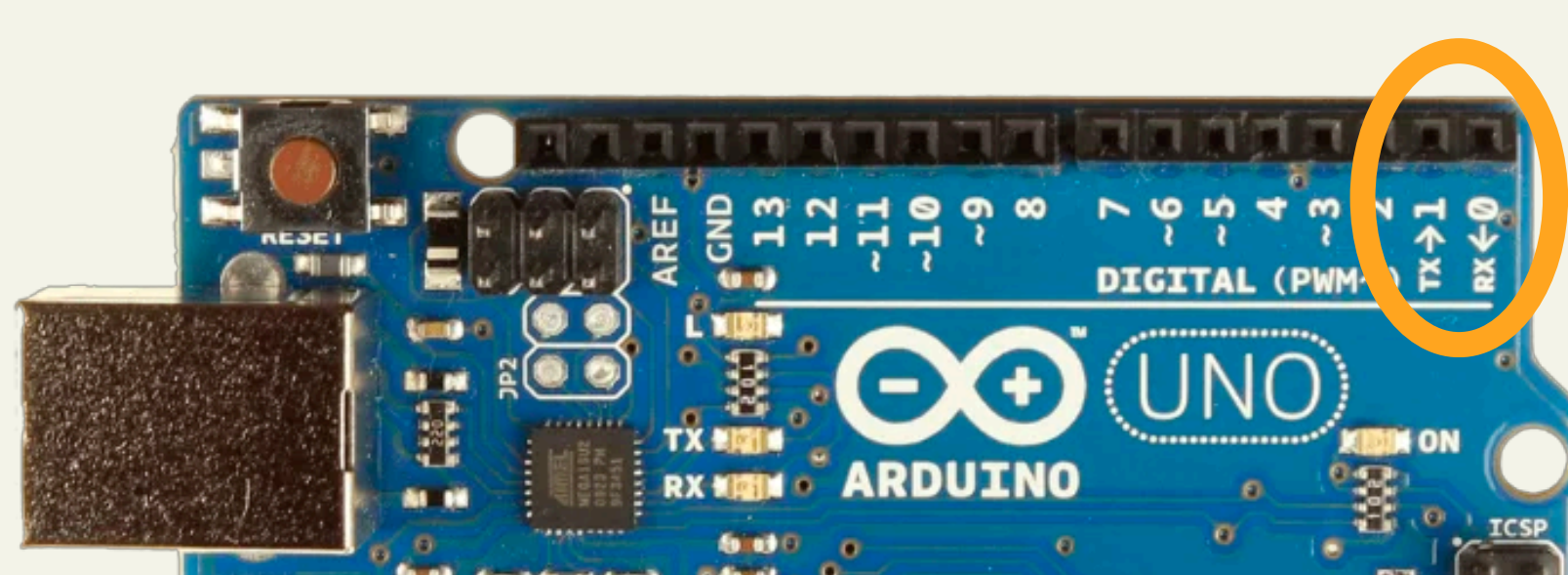
```
tipo_da_variavel dado = Serial.read(); // O valor enviado deve ser armazenado em uma variável.
```

EXEMPLO

```
1 void setup() {  
2     Serial.begin(9600); // Inicia a serial a 9600 bps  
3 }  
4  
5 void loop() {  
6     // Verifica se existe dado chegando na serial  
7     if (Serial.available() > 0) {  
8         // Lê o dado recebido  
9         char dadoRecebido = Serial.read();  
10  
11         // Devolve a mensagem ao usuário  
12         Serial.print("Voce digitou: ");  
13         Serial.println(dadoRecebido);  
14     }  
15 }
```

PORTAS TX E RX NA COMUNICAÇÃO SERIAL

- **TX (Transmissão) e RX (Recepção):** Papéis fundamentais na comunicação serial.
 - **TX:** Transmite dados de um dispositivo.
 - **RX:** Recebe dados em outro dispositivo.
- **Comunicação Bidirecional:**
 - Permite troca eficiente de dados entre dispositivos.
 - O TX de um conecta ao RX do outro (e vice-versa).

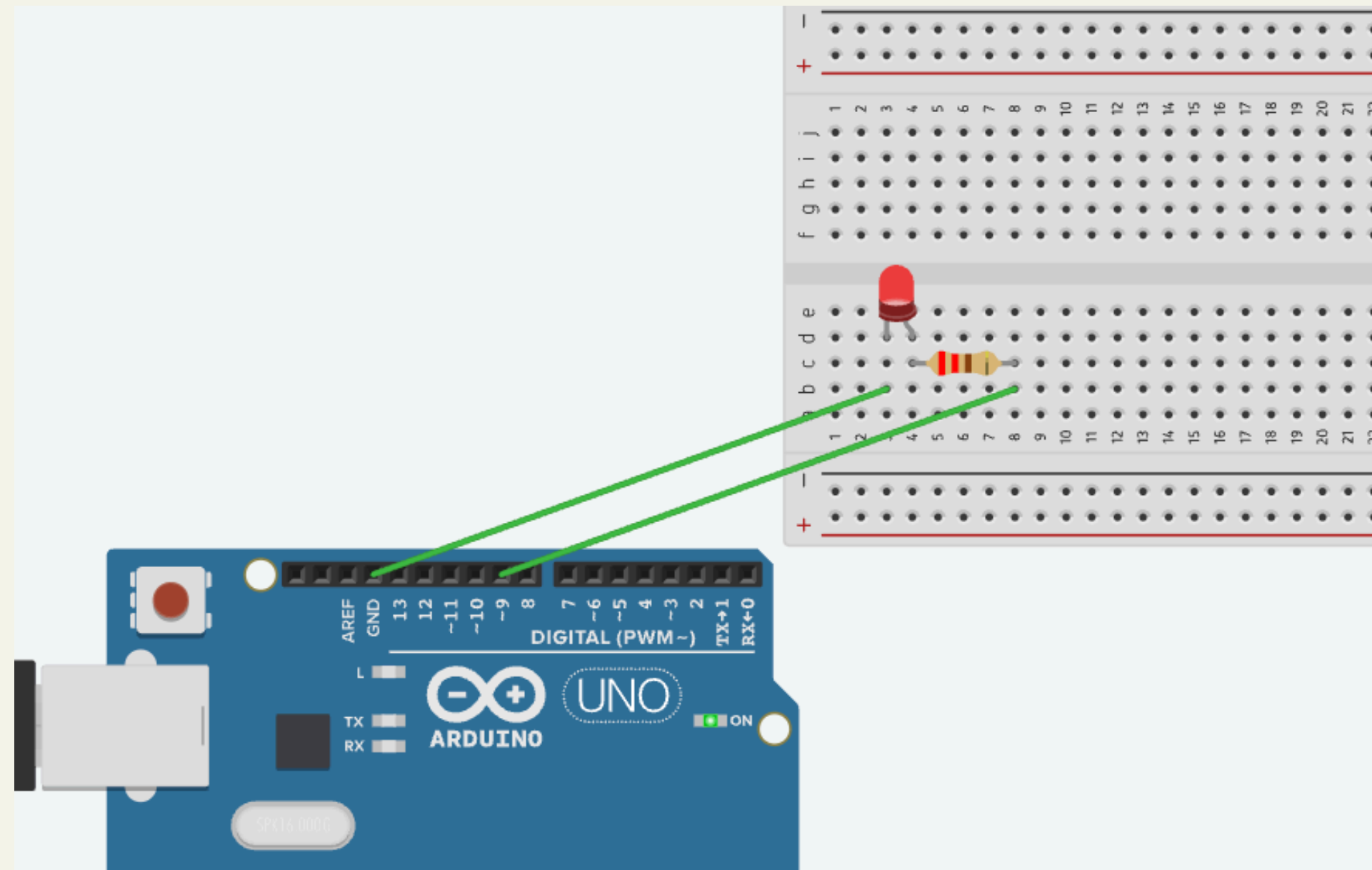


Pinos PWM:
0 e 1

LABORATÓRIO 1: CONTROLE DE LED POR MEIO DA COMUNICAÇÃO SERIAL

- **Configure a comunicação serial de seu arduino, para que, utilizando o serial monitor, consiga controlar um LED.**

LABORATÓRIO 1: CONTROLE DE LED POR MEIO DA COMUNICAÇÃO SERIAL



LABORATÓRIO 1: CONTROLE DE LED POR MEIO DA COMUNICAÇÃO SERIAL

```
1 // Definição dos pinos
2 const int ledPin = ____; // Pino do LED
3
4 void setup() {
5     // Configura o pino do LED como saída
6     pinMode(ledPin, ____);
7     // Inicia a comunicação serial
8     Serial.begin(____);
9     // Mensagem inicial
10    Serial.println("Digite L para ligar ou D para desligar");
11 }
```

```
13 void loop() {
14     // Verifica se chegou algum dado pela serial
15     if (Serial.available() > 0) {
16         // Lê o caractere recebido
17         char comando = Serial.____();
18
19         // Processa o comando
20         if (comando == 'L' || comando == 'l') {
21             digitalWrite(ledPin, ____);
22             Serial.println("LED Ligado!");
23         }
24         else if (comando == 'D' || comando == 'd') {
25             digitalWrite(ledPin, ____);
26             Serial.println("LED Desligado!");
27         }
28         else {
29             Serial.println("Comando inválido! Digite L ou D");
30         }
31     }
32 }
```

LABORATÓRIO 1: CONTROLE DE LED POR MEIO DA COMUNICAÇÃO SERIAL

```
1  const int ledPin = 9;
2
3  void setup() {
4      pinMode(ledPin, OUTPUT);
5      Serial.begin(9600);
6      Serial.println("Digite L para ligar ou D para desligar");
7  }
8
9  void loop() {
10     if (Serial.available() > 0) {
11         char comando = Serial.read();
12
13         if (comando == 'L' || comando == 'l') {
14             digitalWrite(ledPin, HIGH);
15             Serial.println("LED Ligado!");
16         }
17         else if (comando == 'D' || comando == 'd') {
18             digitalWrite(ledPin, LOW);
19             Serial.println("LED Desligado!");
20         }
21         else {
22             Serial.println("Comando inválido! Digite L ou D");
23         }
24     }
25 }
```

BIBLIOTECAS

- **Conjuntos de código pré-escrito que facilitam o uso de componentes e funcionalidades específicas.**
- **Benefícios das Bibliotecas:**
 - **Economia de tempo**
 - **Padronização**
 - **Facilidade (funções prontas para tarefas complexas)**

```
// 1. Incluir a biblioteca (no topo do código)
#include <NomeDaBiblioteca.h>

// 2. Criar objeto (se necessário)
NomeDaBiblioteca objeto;

// 3. Usar funções da biblioteca
objeto.funcao();
```

TIPOS DE BIBLIOTECAS NO ARDUINO

- **Bibliotecas Padrão (Built-in)**
 - Já vêm com o Arduino IDE
 - Exemplo de inclusão no código: `#include <Servo.h>`
- **Bibliotecas de Terceiros**
 - Baixadas da internet, criadas pela comunidade ou por fabricantes
 - Normalmente usadas para sensores e displays
- **Bibliotecas Customizadas**
 - Você mesmo pode criar suas próprias bibliotecas
 - Indicadas para quando você repete o mesmo código em vários projetos (trabalha sempre com o mesmo conjunto de funções).
 - Exemplo de inclusão no código: `#include "minhaBiblioteca.h"`

COMO INSTALAR BIBLIOTECAS

- **Método 1: Pelo Gerenciador de Bibliotecas da IDE**
 - 1) **Vá em Sketch -> Incluir bibliotecas -> Gerenciar Bibliotecas**
 - 2) **Busque o nome da biblioteca desejada**
 - 3) **Clique em instalar**
- **Método 2: Manualmente, por meio de arquivos .zip**
 - 1) **Baixe a biblioteca na internet(arquivo .zip)**
 - 2) **Vá em Sketch -> Incluir bibliotecas -> Adicionar Biblioteca .ZIP**
 - 3) **Selecione o arquivo baixado**

DISPLAY LCD

- **LCD = Liquid Crystal Display (Display de Cristal Líquido)**
- **Definição: Componente eletrônico que exibe texto e números usando cristais líquidos que bloqueiam ou permitem a passagem de luz.**
- **Tipos Comuns para Arduino:**
 - **16x2: Mostra 32 caracteres no total**
 - **20x4: Mostra 80 caracteres no total**

DISPLAY LCD 16X2

- Destaque para iniciantes devido à versatilidade.
- Configuração de Pinos:
 - 16 pinos, com 12 na conexão básica.
 - 11 de entrada/saída, 2 de alimentação, 2 de controle de backlight e contraste.

Característica	Valor típico
Tensão de operação	5 V
Corrente de operação	~1-2 mA (<i>sem iluminação</i>)
Área do visor	~64,5 × 13,8 mm
Tamanho do display	16 colunas × 2 linhas

DISPLAY LCD 16X2



TECNOLOGIA DE CRISTAL LÍQUIDO

- **O que é cristal líquido?**
 - Estado da matéria entre sólido e líquido
 - Tem organização como sólido
 - Flui como líquido
 - Reage à eletricidade

COMO O DISPLAY LCD FUNCIONA?

- **O LCD não produz luz, ele apenas controla a luz. Possui 3 camadas:**
- **1. Luz de Fundo (Backlight)**
 - Luz branca atrás do display
 - Fica sempre ligada
- **2. Cristal Líquido**
 - Controla a passagem da luz
 - Com eletricidade: bloqueia a luz (escuro)
 - Sem eletricidade: deixa a luz passar (claro)
- **3. Filtro de Cor**
 - Define a cor do fundo e das letras

FUNÇÕES DOS PINOS

- **Os displays LCD 16x2 geralmente possuem 16 pinos, cada um com uma função específica. Aqui estão as funções dos pinos mais comuns:**
 - **VSS (Ground):** Conectado ao terra (GND) do circuito
 - **VDD (Power Supply):** Conectado à alimentação positiva (VCC)
 - **VO (Contrast):** Usado para ajustar o contraste do display (potenciometro)
 - **RS (Register Select):** Define se os dados fornecidos aos pinos D0-D7 são dados ou comandos. RS=0 (LOW) para comandos, RS=1 (HIGH) para dados.
 - **RW (Read/Write):** Define se a operação é de leitura (RW=1) ou escrita (RW=0). Muitas vezes, mantido em GND para operação de escrita.
 - **E (Enable):** Ativa a leitura ou escrita de dados quando transita de 0 para 1.

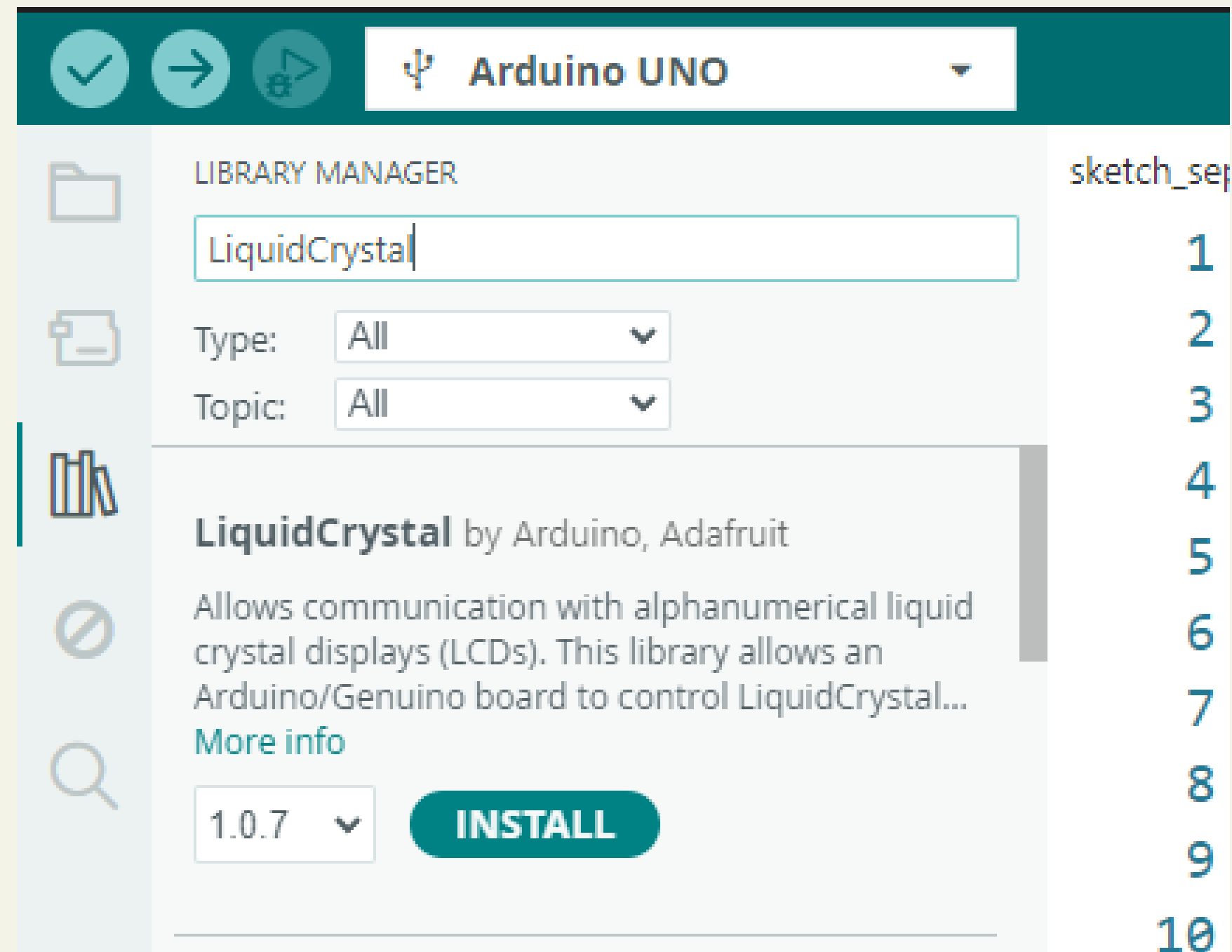
FUNÇÕES DOS PINOS

- **Os displays LCD 16x2 geralmente possuem 16 pinos, cada um com uma função específica. Aqui estão as funções dos pinos mais comuns:**
 - **D0-D7 (Data Lines):** Linhas de dados utilizadas para enviar dados ou comandos ao display. A quantidade de linhas (4 ou 8) depende da configuração do display.
 - **A (Anode):** Alimentação positiva do backlight (luz de fundo).
 - **K (Cathode):** Conectado ao terra (GND) para fechar o circuito do backlight.
 - **BLA e BLK (Backlight Anode e Backlight Cathode):** Variações dos pinos A e K, são usados para alimentação do backlight em alguns modelos.

BIBLIOTECAS PARA DISPLAY LCD 16X2

- **LiquidCrystal Library:**
 - Biblioteca padrão no ambiente de desenvolvimento do Arduino.
 - Oferece funções básicas para controlar displays LCD, incluindo o 16x2.
- **HD44780 Library:**
 - Implementação moderna do controlador HD44780.
 - Recursos adicionais e otimizações em comparação com a LiquidCrystal Library.
 - Instalação direta pelo Gerenciador de Bibliotecas do Arduino.
- **LCD I2C Library:**
 - Projetada para displays LCD com interface I2C.
 - Simplifica comunicação com displays 16x2 que possuem adaptadores I2C.
 - Instalação direta pelo Gerenciador de Bibliotecas do Arduino.

INSTALANDO A BIBLIOTECA



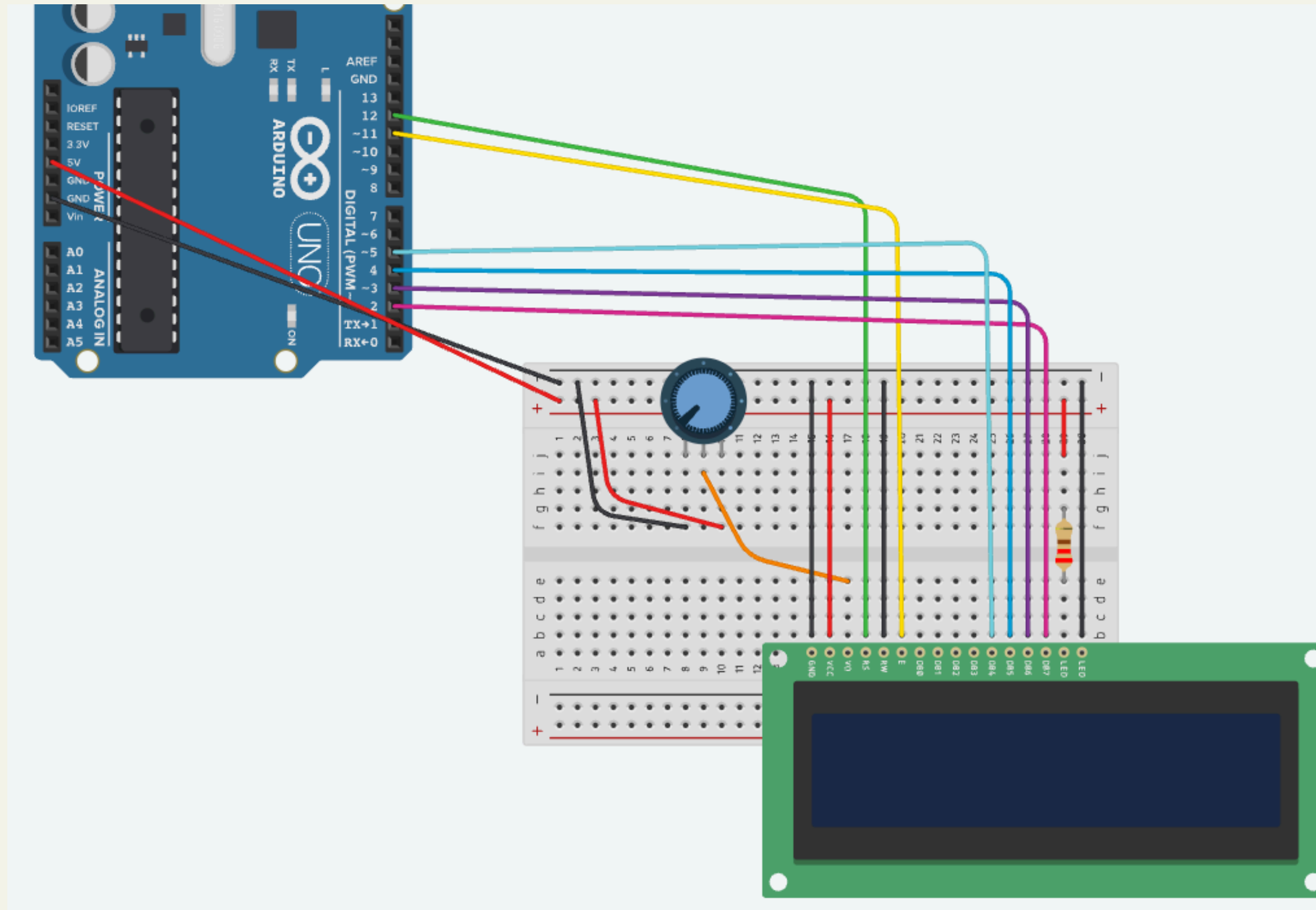
FUNÇÕES DA BIBLIOTECA LIQUIDCRYSTAL

- **LiquidCrystal():**
 - Construtor para criar uma instância da classe 'LiquidCrystal'. É preciso definir os pinos de controle do seu display no construtor.
- **begin(cols, rows):**
 - Inicializa o display com o número de colunas e linhas especificado.
- **clear():**
 - Limpa o conteúdo do display.
- **home():**
 - Move o cursor para a posição inicial (1ª coluna, 1ª linha).
- **setCursor(col, row):**
 - Define a posição do cursor no display.

FUNÇÕES DA BIBLIOTECA LIQUIDCRYSTAL

- **print(data):**
 - Exibe texto ou dados no display. Pode receber strings, números, caracteres, etc.
- **write(character):**
 - Escreve um único caractere no display.
- **scrollDisplayLeft() e scrollDisplayRight():**
 - Rola o conteúdo do display para a esquerda ou direita.
- **blink() e noBlink():**
 - Ativa ou desativa o modo de piscar o cursor.
- **cursor() e noCursor():**
 - Ativa ou desativa a exibição do cursor.
- **createChar():**
 - Define padrões de pixels específicos que serão utilizados para criar caracteres personalizados.

CONEXÃO NO ARDUINO



EXEMPLO

```
1  #include <LiquidCrystal.h>
2
3  LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
4
5  void setup() {
6      lcd.begin(16, 2);
7
8      lcd.print("Ola!");
9      lcd.setCursor(0, 1);
10     lcd.print("Arduino");
11 }
12
13 void loop() {
14     delay(2000);
15
16     lcd.clear();
17     lcd.setCursor(0, 0);
18     lcd.print("LCD 16x2");
19
20     delay(2000);
21 }
```

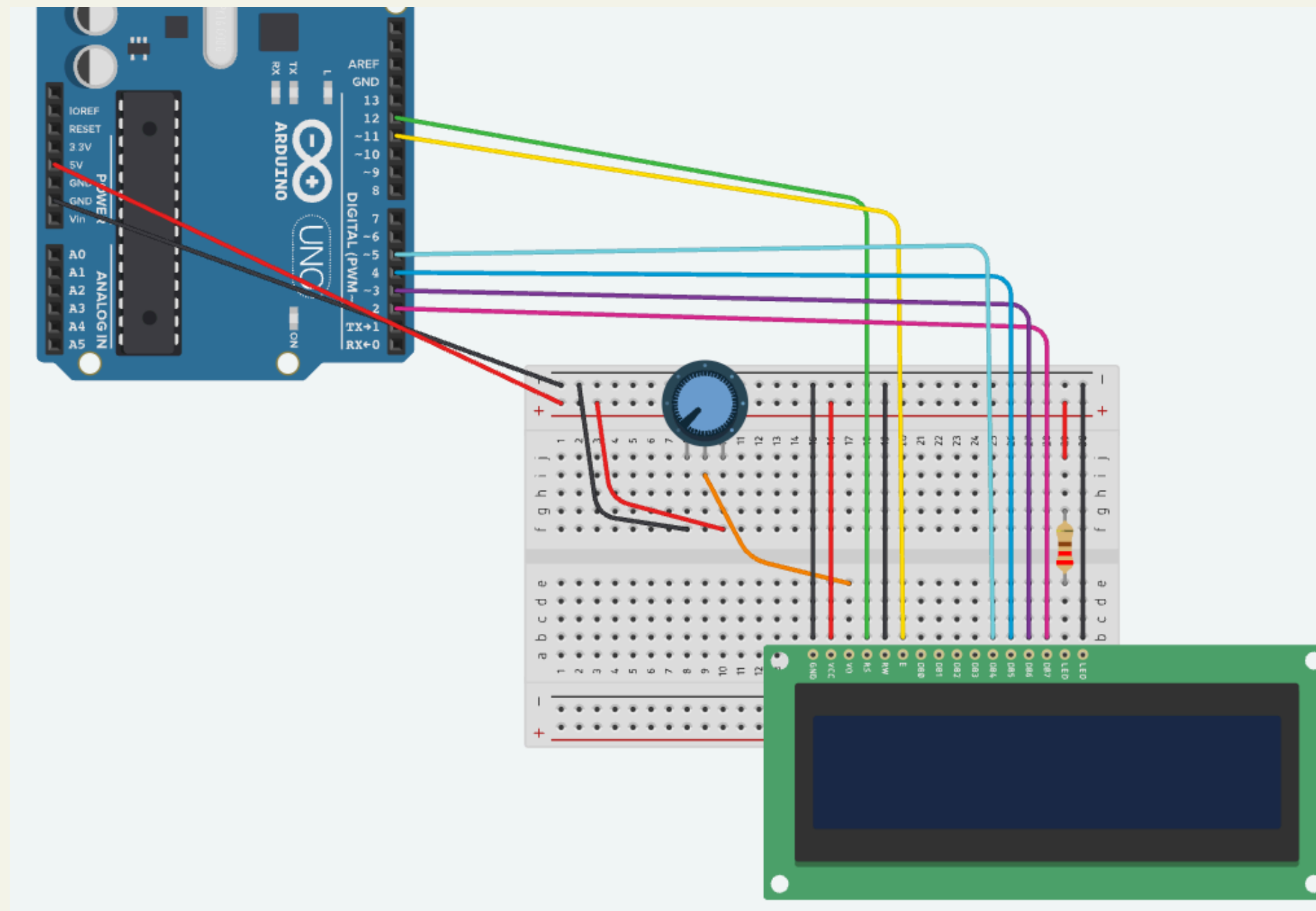
EXEMPLO

```
1  #include <LiquidCrystal.h>
2
3  LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
4
5  byte coracao[8] = {
6      0b00000,
7      0b01010,
8      0b11111,
9      0b11111,
10     0b01110,
11     0b00100,
12     0b00000,
13     0b00000
14 };
15
16 void setup() {
17     lcd.begin(16, 2);
18
19     lcd.createChar(0, coracao);
20
21     lcd.setCursor(0, 0);
22     lcd.print("Eu ");
23     lcd.write(byte(0));
24     lcd.print(" Arduino");
25 }
```

LABORATÓRIO 2: ESCREVENDO NO DISPLAY LCD

- **Configure o circuito do display LCD e faça um código que utilize a comunicação serial para escrever o que for mandado em sua tela.**

LABORATÓRIO 2: ESCREVENDO NO DISPLAY LCD



LABORATÓRIO 2: ESCREVENDO NO DISPLAY LCD

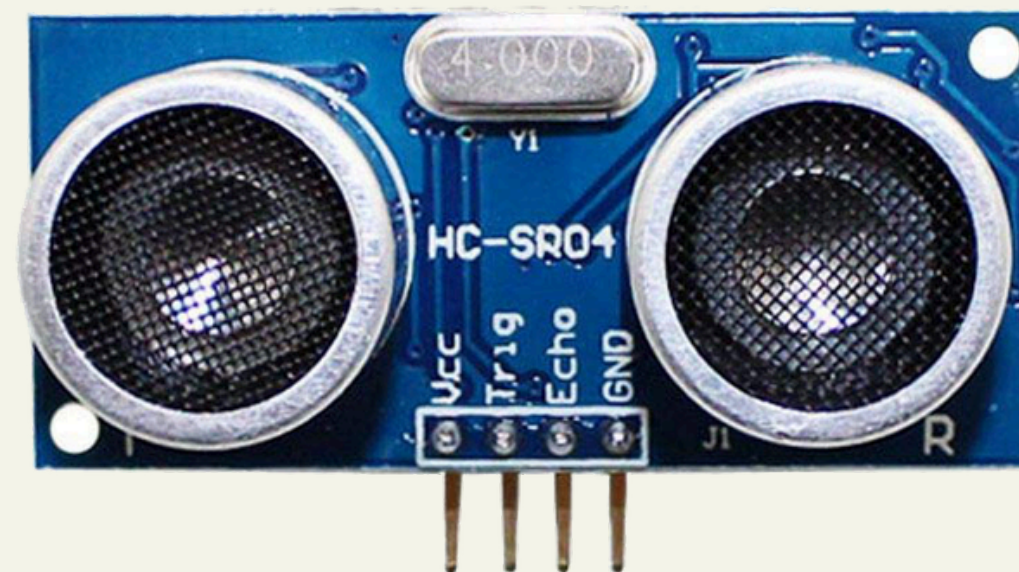
```
1  #include <LiquidCrystal.h>
2
3  LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
4
5  void setup() {
6      lcd.begin(16, 2);
7      lcd.print("Ola, Arduino!");
8      Serial.begin(9600);
9  }
10
11 void loop() {
12     if (Serial.available() > 0) {
13         lcd.clear();
14
15         // Lê a string inteira até o tempo limite (timeout)
16         String mensagem = Serial.readString();
17
18         // Remove espaços em branco ou quebras de linha no final
19         mensagem.trim();
20
21         lcd.setCursor(0, 0);
22         lcd.print(mensagem);
23
24         delay(2000);
25         lcd.clear();
26     }
27 }
```

SENSOR ULTRASSÔNICO

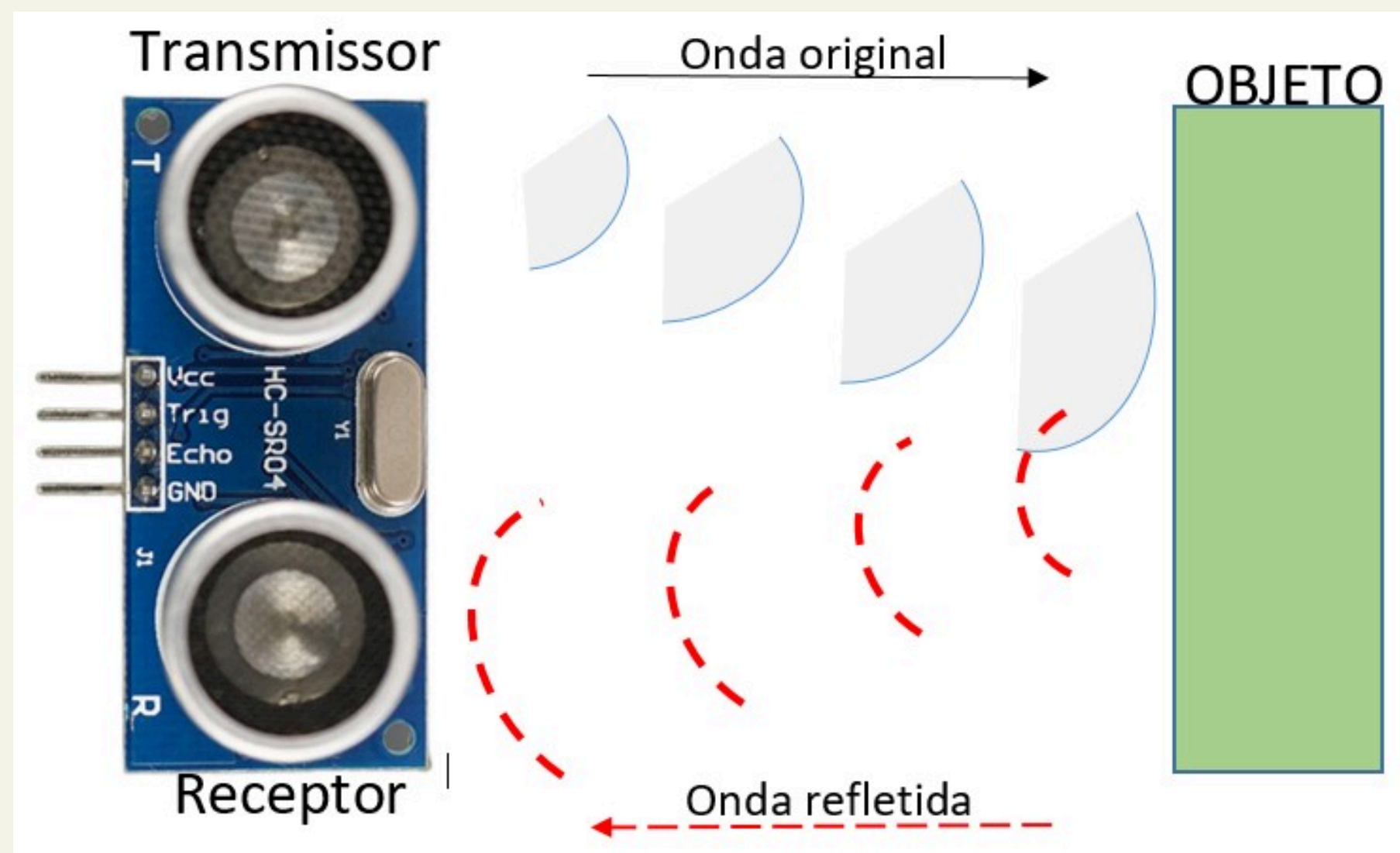
- **O sensor ultrassônico é um dispositivo utilizado para medir a distância até um objeto por meio da emissão e recepção de ondas ultrassônicas.**
- **Emite uma onda sonora em uma frequência acima da faixa audível pelo ser humano.**
- **A onda atinge um objeto e é refletida.**
- **O sensor mede o tempo entre a emissão e o recebimento do eco.**
- **A distância é calculada a partir desse tempo.**

SENSOR ULTRASSÔNICO

- **Sensor HC-SR04 é um dos sensores ultrassônicos mais utilizados com Arduino.**
- **Possui quatro pinos:**
 - **VCC:** alimentação de 5 V.
 - **GND:** referência do circuito.
 - **TRIG (Trigger):** recebe o sinal para iniciar a medição.
 - **ECHO:** fornece um pulso cuja duração representa o tempo de retorno do sinal.



FUNCIONAMENTO DO SENSOR ULTRASSÔNICO



SENSOR ULTRASSÔNICO

- A distância é obtida a partir do tempo de viagem da onda:

$$D = \frac{t \times v}{2}$$

Onde:

D → distância até o objeto

t → tempo medido pelo ECHO

v → velocidade do som (aproximadamente 0,0343 cm/μs)

2 → percurso de ida e volta

SENSOR ULTRASSÔNICO

```
digitalWrite(trigPin, LOW);  
delayMicroseconds(2);
```

```
digitalWrite(trigPin, HIGH);  
delayMicroseconds(10);
```

```
digitalWrite(trigPin, LOW);
```

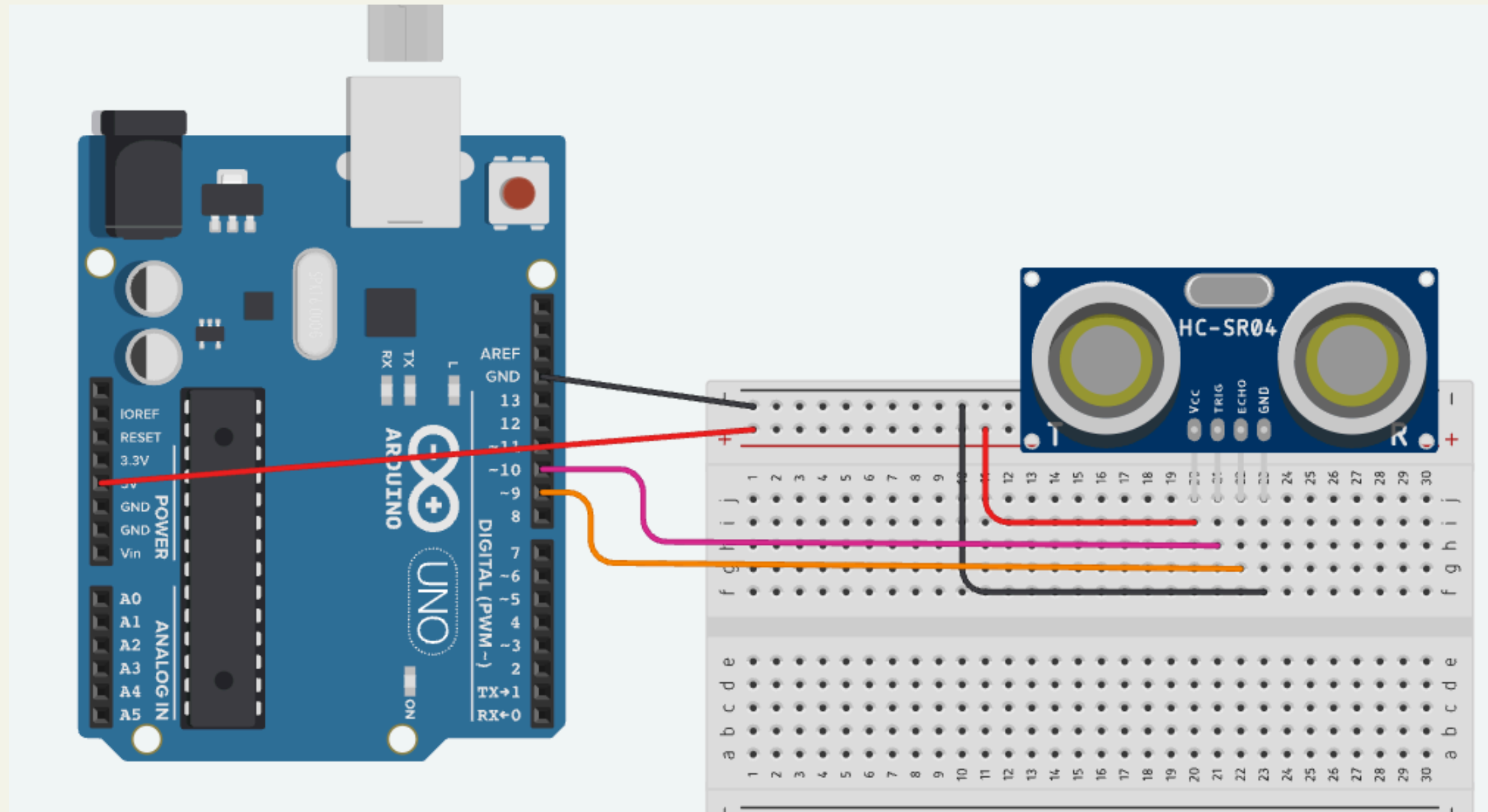
SENSOR ULTRASSÔNICO

```
float duration = pulseIn(echoPin, HIGH);  
  
float distance = (duration * 0.0343) / 2;  
  
Serial.print("Distance: ");  
Serial.println(distance);
```

LABORATÓRIO 3: ESCREVENDO NO MONITOR A DISTANCIA DE UM OBJETO

- **Utilize um sensor ultrassônico para medir a distancia de um objeto e imprima ele na tela em seu monitor serial**

LABORATÓRIO 3: ESCREVENDO NO MONITOR A DISTANCIA DE UM OBJETO



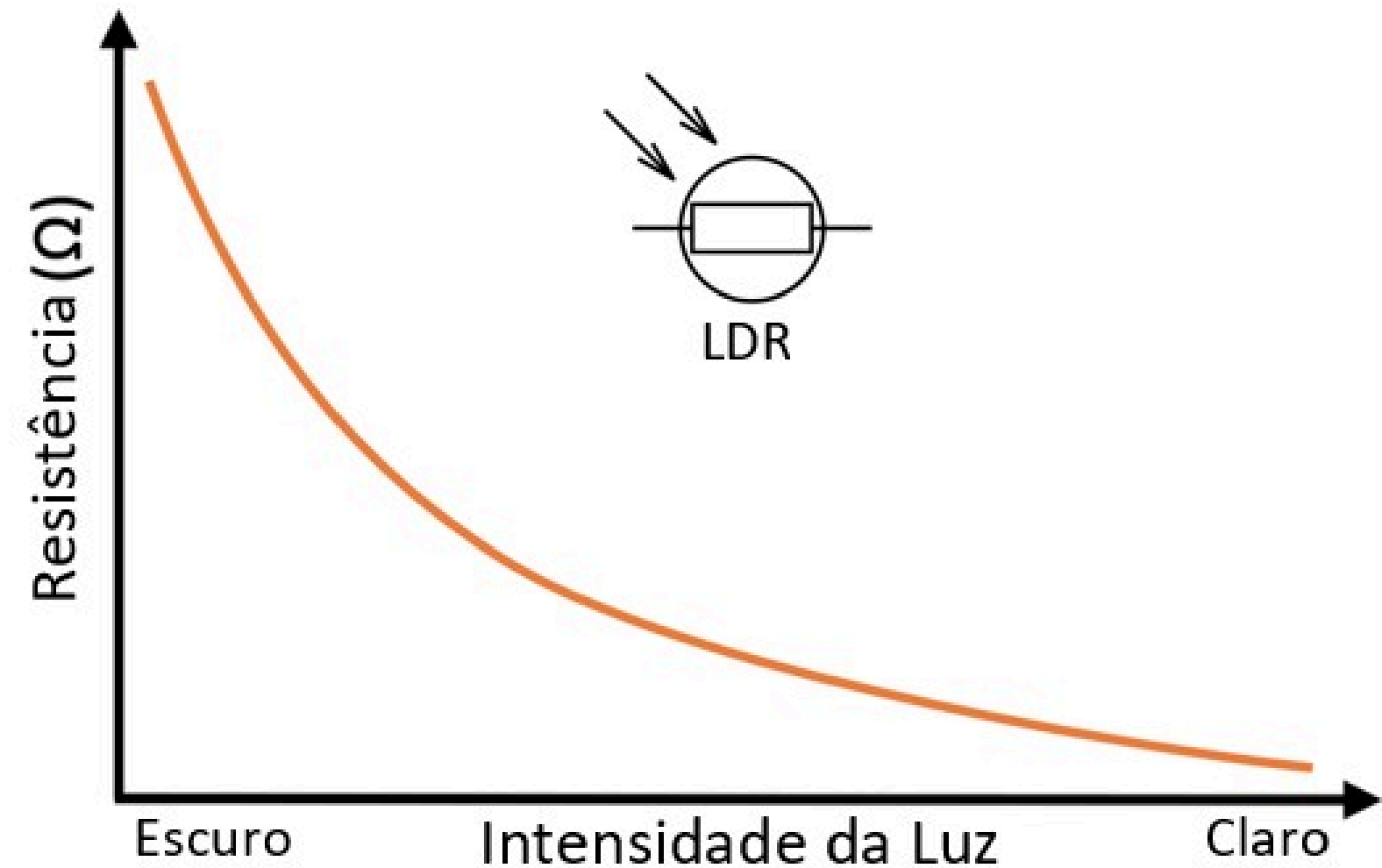
LABORATÓRIO 3: ESCREVENDO NO MONITOR A DISTANCIA DE UM OBJETO

```
1  const int trigPin = 10;
2  const int echoPin = 9;
3
4  void setup() {
5      pinMode(trigPin, OUTPUT);
6      pinMode(echoPin, INPUT);
7      Serial.begin(9600);
8  }
9
10 void loop() {
11     digitalWrite(trigPin, LOW);
12     delayMicroseconds(2);
13     digitalWrite(trigPin, HIGH);
14     delayMicroseconds(10);
15     digitalWrite(trigPin, LOW);
16
17     float duration = pulseIn(echoPin, HIGH);
18     float distance = (duration*.0343)/2;
19     Serial.print("Distance: ");
20     Serial.println(distance);
21     delay(100);
22 }
```

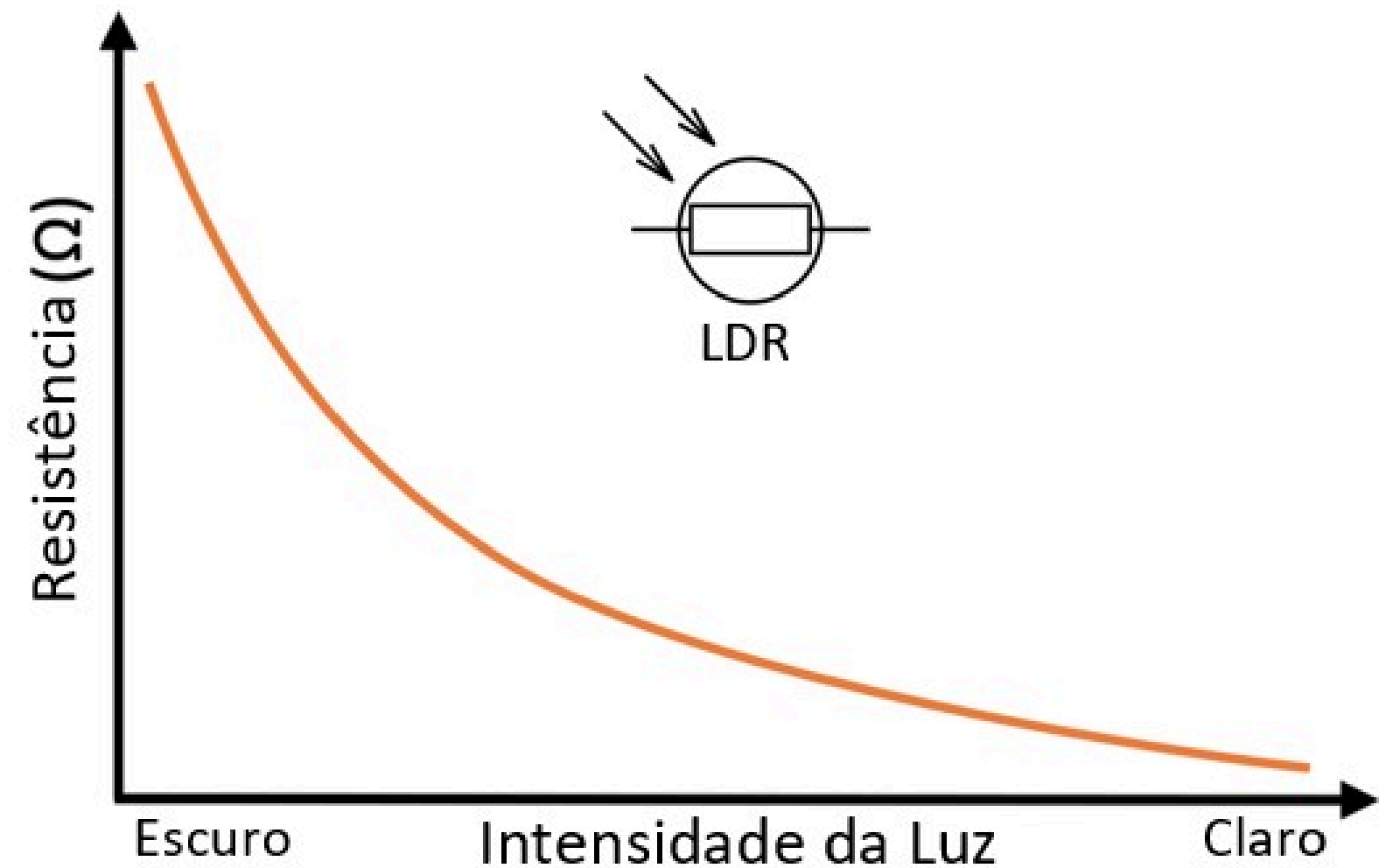
SENSOR LDR (LIGHT DEPENDENT RESISTOR)

- **Definição:**
 - **Sensor eletrônico sensível à luz cuja resistência varia conforme a intensidade luminosa.**
- **Configuração:**
 - **A resistência diminui em ambientes iluminados e aumenta na ausência de luz.**
- **Conexão com Arduino:**
 - **Realiza leituras analógicas e deve ser conectado a portas analógicas (A0–A5).**
 - **A conversão digital varia de 0 a 1023, onde 0 indica ausência de luz e 1023 alta intensidade luminosa.**

SENSOR LDR (LIGHT DEPENDENT RESISTOR)



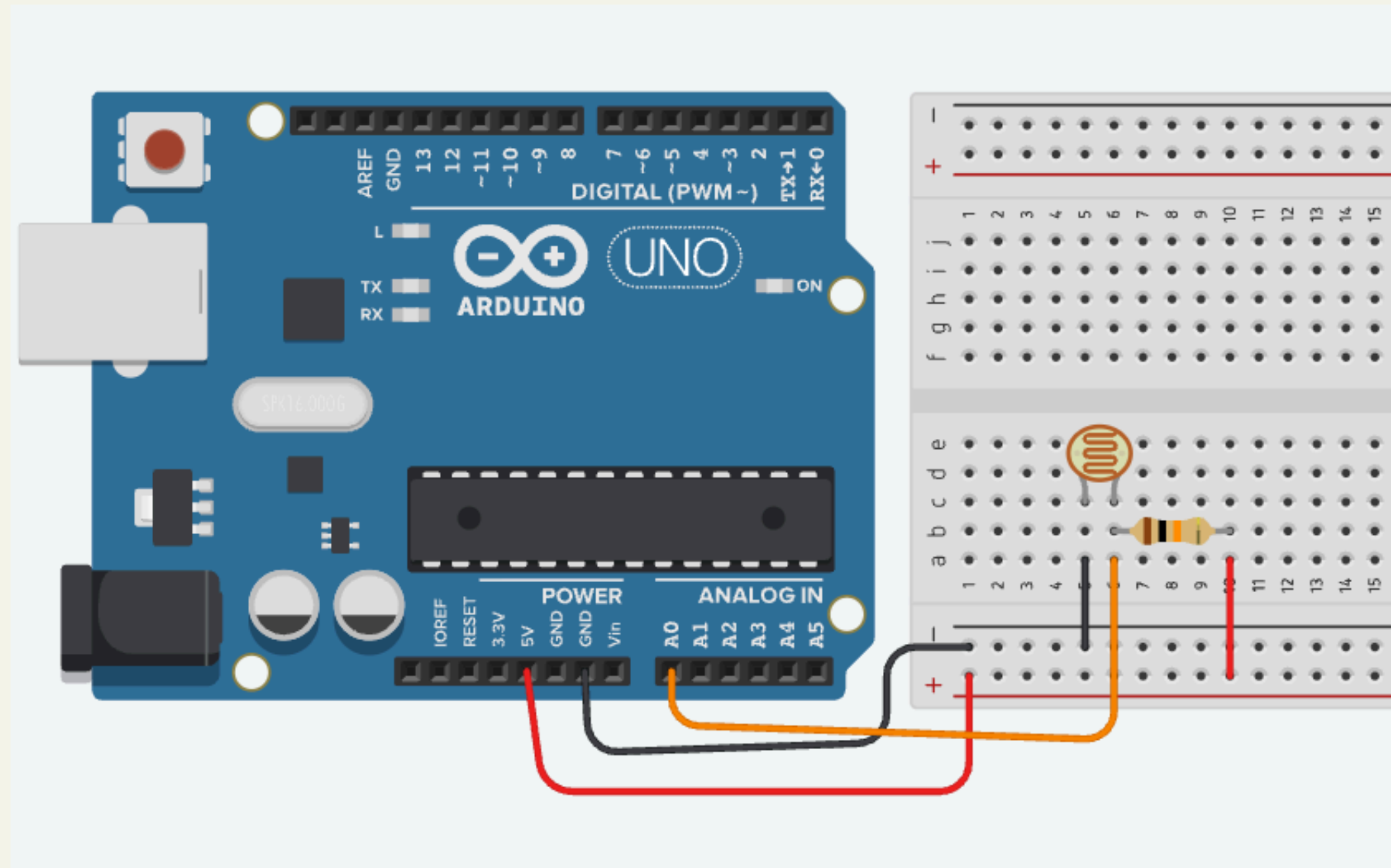
SENSOR LDR (LIGHT DEPENDENT RESISTOR)



LABORATÓRIO 4: MEDINDO LUMINOSIDADE

- **Utilize um sensor LDR para medir a luminosidade do ambiente e imprima o valor na tela em seu monitor serial**

LABORATÓRIO 4: MEDINDO LUMINOSIDADE



LABORATÓRIO 4: MEDINDO LUMINOSIDADE

```
1 //declaração do pino e varável que armazena a leitura
2 int LDR = A0;
3 int leitura = 0;
4
5 void setup(){
6     pinMode(LDR, INPUT);
7     //Como LDR é um sensor ele irá captar informações logo é declarado como INPUT
8     Serial.begin(9600);
9 }
10
11 void loop(){
12     leitura = analogRead(LDR);
13     Serial.print("Valor lido pelo LDR: ");
14     Serial.println(leitura);
15     Delay(100);
16 }
```

SERVOMOTOR

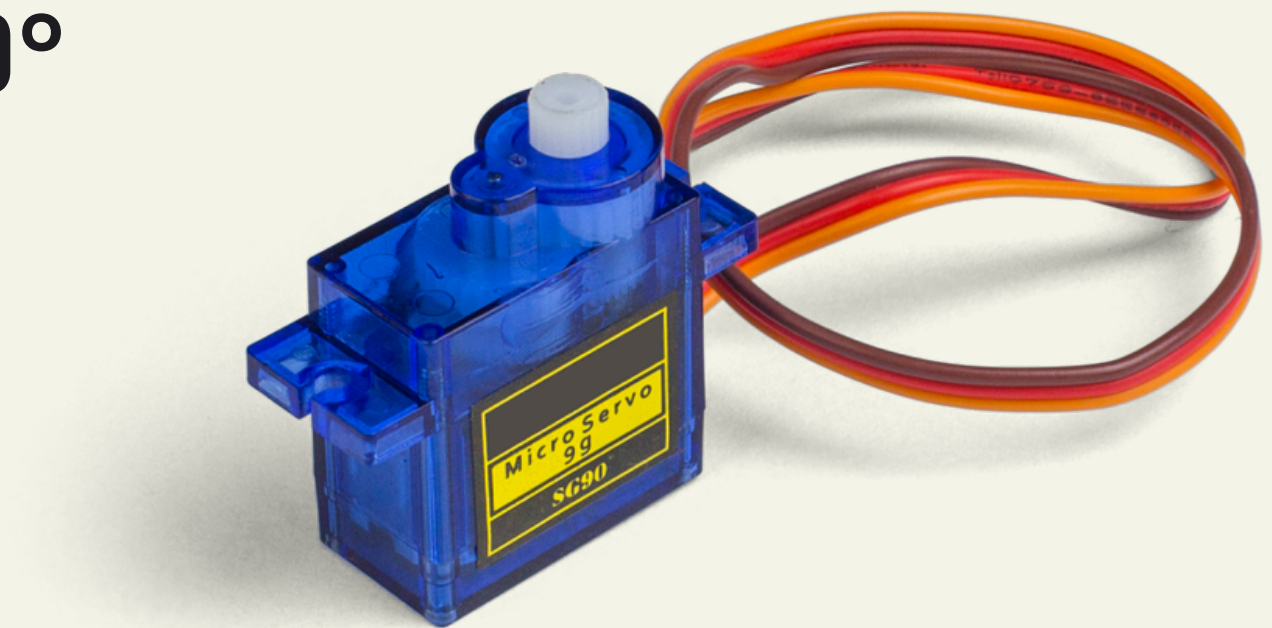
- **O servomotor é um dispositivo capaz de controlar precisamente a posição de um eixo.**
- **Diferentemente de um motor DC comum, que normalmente gira continuamente, um servomotor pode ser comandado para atingir uma posição angular específica.**
- **Possui um motor elétrico interno.**
- **Utiliza um sistema de controle para determinar a posição do eixo.**
- **O Arduino envia um sinal de controle para determinar a posição desejada.**
- **É muito utilizado em sistemas que precisam de movimentos precisos.**

SERVOMOTOR

- **O servomotor possui internamente:**
 - **Motor DC → realiza o movimento.**
 - **Engrenagens → aumentam o torque e reduzem a velocidade.**
 - **Sensor de posição → identifica a posição atual do eixo.**
 - **Circuito de controle → compara a posição atual com a posição desejada.**

SERVOMOTOR SG90

- **O SG90 é um pequeno servomotor muito utilizado em projetos educacionais e de prototipagem.**
- **Características típicas**
 - **Tensão de operação: aproximadamente 4,8–6 V**
 - **Faixa angular: aproximadamente 0° a 180°**
 - **3 fios:**
 - **VCC → alimentação**
 - **GND → terra**
 - **Sinal → controle**



BIBLIOTECA SERVO

✓

→

⚙️

Arduino UNO

LIBRARY MANAGER

servo

Type: All

Topic: All

Servo by Michael Margolis, Arduino

Allows Arduino boards to control a variety of servo motors. This library can control a great number of servos.
It makes careful use of...
More info

1.3.0

INSTALL

sketch_sep1

1

2

3

4

5

6

7

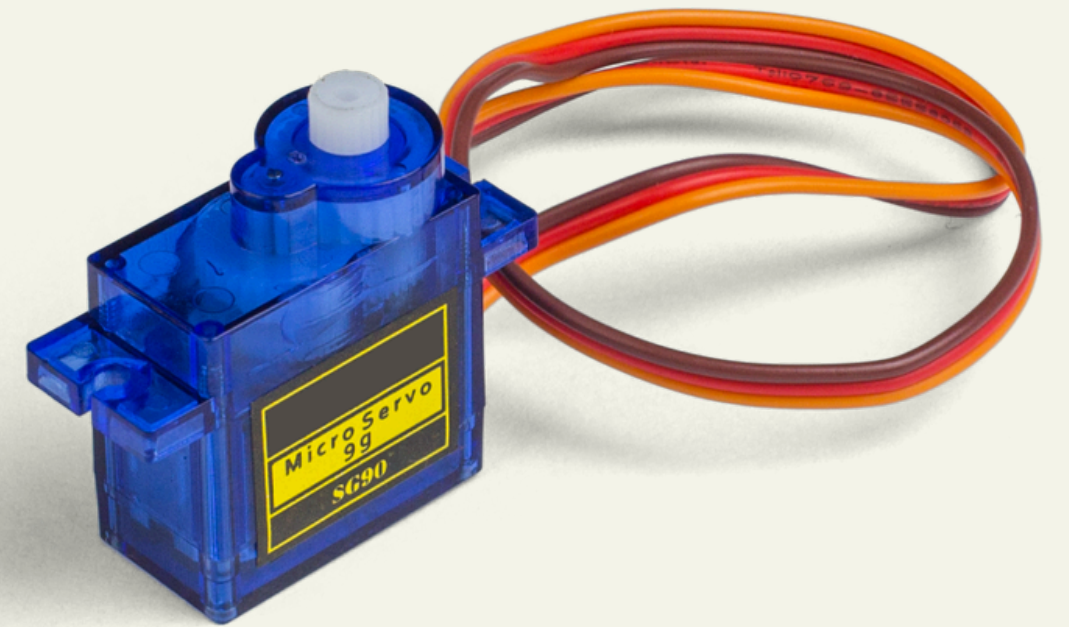
8

9

10

BIBLIOTECA SERVO

- **Servo meuServo;**
 - **Cria um objeto para representar o servomotor.**
- **attach(pino)**
 - **Define o pino utilizado para o sinal do servo.**
- **write(angulo)**
 - **Define a posição angular desejada.**
- **detach()**
 - **Interrompe o controle do servo pelo pino.**



EXEMPLO

```
1  #include <Servo.h>
2
3  Servo meuServo;
4
5  void setup() {
6      meuServo.attach(9);
7  }
8
9  void loop() {
10     meuServo.write(0);
11     delay(1000);
12
13     meuServo.write(90);
14     delay(1000);
15
16     meuServo.write(180);
17     delay(1000);
18 }
```

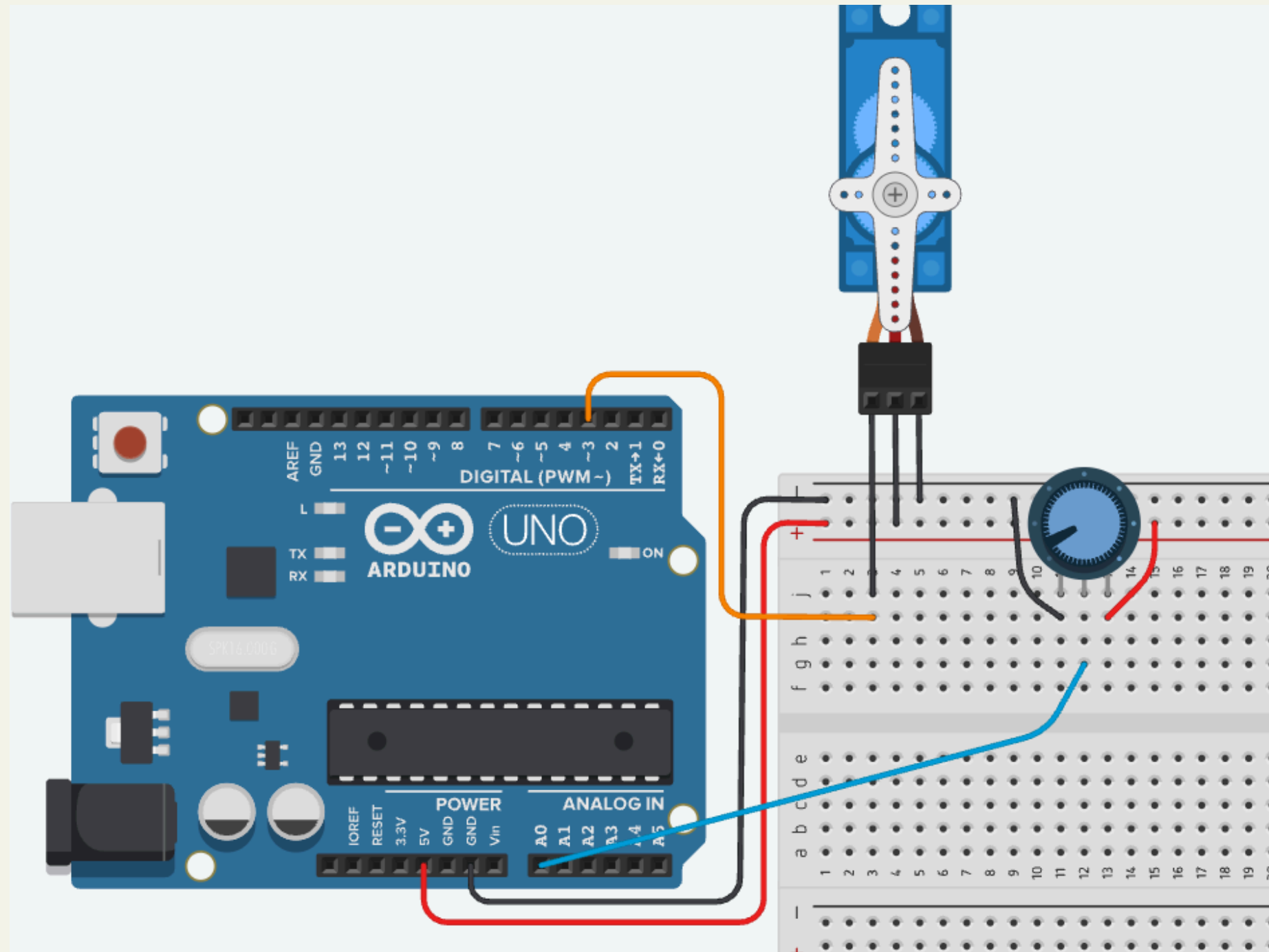
EXEMPLO

```
1  #include <Servo.h>
2
3  Servo meuServo;
4
5  void setup() {
6      meuServo.attach(9);
7  }
8
9  void loop() {
10     meuServo.write(0);
11     delay(1000);
12
13     meuServo.write(90);
14     delay(1000);
15
16     meuServo.write(180);
17     delay(1000);
18 }
```

LABORATÓRIO 5: SERVOMOTOR COM POTENCIÔMETRO

- **Utilize um potenciômetro para controlar a posição de um servomotor. Para facilitar, pode ser utilizada a função `map()` para converter o sinal do potenciômetro para a abrangência do ângulo do servomotor.**

LABORATÓRIO 5: SERVOMOTOR COM POTENCIÔMETRO



LABORATÓRIO 5: SERVOMOTOR COM POTENCIÔMETRO

```
1  #include <Servo.h>
2
3  Servo meuServo;
4
5  int potenciometro = A0;
6  int leitura;
7  int angulo;
8
9  void setup() {
10     pinMode(potenciometro, INPUT);
11     meuServo.attach(3);
12 }
13
14 void loop() {
15     leitura = analogRead(potenciometro);
16
17     angulo = map(leitura, 0, 1023, 0, 180);
18
19     meuServo.write(angulo);
20 }
```

**MUITO OBRIGADA PELA
ATENÇÃO!**

